# 姓名：徐振瑞

电话：16608682271

邮箱：shironyaa@qq.com

出生年月：2001年11月（23岁）

## 🎓教育背景

**2018年9月 ~ 2022年7月** **天津大学** **自动化（本科）**

## 💼工作经历

**2021年10月 ~ 2024年10月** **最右（迅雷/小川科技）** **机器学习平台开发工程师**

**工作职责**：负责机器学习平台相关工作，推荐模型的训练/推理及特征服务、大模型训练/推理平台建设

## 🛠专业技能

- 熟悉 **推荐模型** 的训练、推理、特征工程相关工作，参与过推荐模型 **分布式训练框架** 的开发
- 熟悉 **大模型** 相关知识，了解 **DeepSpeed、vLLM** 等大模型框架，了解 **SFT、LORA** 等微调方法
- 熟悉 **LangChain** 等大模型相关工具，了解 **RAG** 相关技术，参与过大模型 **Agent** 平台的开发
- 熟悉 **Golang、Python** 等编程语言，参与过推荐系统相关工程与大模型相关工程的开发

## 🚀工作项目

**一、推荐模型分布式训练框架 PServer**

**项目背景：**为了加快大型推荐模型的训练速度，同时探索实时推荐模型的技术路线，需要使用分布式训练框架，以提供分布式训练与模型参数实时更新等功能。

**项目内容：**使用参数服务器的分布式训练架构，每台机器上存在一个Server节点和一个Worker节点，各节点之间通过MPI进行通信。

- 训练数据被均分以提高训练速度，Worker节点根据数据ID按行从Server节点获取模型参数；
- Server节点均分模型参数，通过一致性哈希算法确定各个Key的存储节点；
- 每训练完一个batch的数据，Worker节点将该节点梯度push到Server节点更新模型参数。

**个人职责：**负责Server模块的开发、各种常见优化器的实现、模型加载/保存的优化

**技术要点：**

- 对于周期性更新的模型，模型检查点的加载与保存使用ProtoBuf进行存储，每个特征的模型参数分开存储并进行分片；这样在进行模型加载时，也可以利用分片机制，多机并行加载参数，提升加载速度；
- 对于实时更新的模型，模型参数存放在Redis中，线上生成的推荐数据每攒够一个Batch便实时更新一次模型参数，在线模型推理服务使用带过期时间的LRU缓存半实时获取模型参数；
- 关于推荐模型使用非AdaGrad优化器时存在的问题：
    - 推荐模型中，参数嵌入矩阵的行，往往代表User/Post的ID（通过哈希表映射），在周期性进行模型训练时，由于矩阵运算的原因，累计梯度值的更新通常是整表更新，这会导致该周期内无训练数据的ID，其对应行中的模型权重的累计梯度也存在额外更新（甚至对于RMSProp和Adam优化器，模型权重也会被更新）。
    - 在PServer训练框架中，通过按行计算，可以避免矩阵运算时优化器参数额外更新带来的问题。

**技术组件：**C++、ProtoBuf、MPI

**二、推荐策略自动化调参服务 EvolutionStrategy**

**项目背景：** 重排模块是推荐系统离用户最近的模块，其参数配置对用户体验至关重要，使用人工调参往往存在一定的不可控性，自动化调参服务ES可以根据用户反馈进行精细化调参。

**项目内容：** Post在经过精排模型进行打分后，图文和视频各取TopN进行多目标融合。

- 初始化各精排模型的权重参数，取其中少部分流量在Base参数附近进行随机扰动（符合高斯分布）；
- 在线服务Dump对照组和实验组的埋点日志，实时拼接落入ClickHouse特征仓库；
- 离线服务根据线上数据计算各样本的Reward，以此确定参数进化方向并进行参数更新。

**个人职责：**负责离线部分的Reward计算与重排参数更新

**技术要点：**

- Reward的公式很大程度上决定了进化策略的成功性，为了便于算法侧进行调整优化，在平台中提供了便捷可视化的Reward配置方式，可以实时通过前端完成Reward公式的修改与调整；
- Reward对重排参数的影响，参考了深度学习反向传播中的优化器，通过样本数据与扰动情况，计算出Reward的梯度后，就可以像神经网络一样对参数进行反馈调节。

**技术组件：**Python、Golang

**三、推荐模型线上特征服务 FeatureServer**

**项目背景：**线上推荐模型进行推理时，特征来源多样且计算逻辑繁杂，需要一个统一的服务整合各个来源的推荐特征，为线上模型推理服务提供全量特征集合。

**项目内容：**服务集成线上模型所需的所有特征，为模型预估返回全量特征集，主要分为两部分模块。

- 特征集成Set模块：提供统一的特征写入接口，实现特征的写入，包括推荐池特征、帖子实时特征、用户统计特征等，其中帖子相关特征实时更新，存入MongoDB，用户统计特征按天更新，存入Redis；
- 特征获取Get模块：特征获取模块，根据请求模型的schema组装对应的特征集合返回，主要通过缓存的方式提升接口的响应速度，通过调整存储结构降低内存压力，提升线上性能。

**个人职责：**负责服务的整体方案设计、Set模块的开发、Get模块的内存与接口耗时优化

**技术要点：**

- Set服务周期性短时间大量更新特征引起MongoDB压力过大的优化：
  - 引入Kafka，所有请求需要经过Kafka排队，再由后端服务控制速度统一写入MongoDB。
- Get服务内存占用高的优化：
  - Mongo特征实时性较强，需要全量读取进内存，使用HashMap内存占用较高，因此可以通过使用特征的ID，将存储结构转换为Array；同时由于存在许多特征位空缺（用户基础特征从推荐上游服务直接获取），可以将存储特征的ID与Array索引进行映射，大幅减轻服务内存压力。
- Get服务接口延时较高且99分位不稳定的优化：
  - 定制特征内存池，避免在进行特征更新时频繁创建/销毁内存空间，减轻GC压力；
  - 用户统计特征量级较大且更新较慢，只能通过Redis进行实时读取，通过LRU缓存机制可以大幅减轻服务和Redis的通信压力，同时后续为了进一步优化缓存命中，设计通过Redis的Pub/Sub机制，在推荐链路前段就可以将用户ID提前告知特征服务，使得99分位延时稳定性大幅提高。

**技术组件：**Golang、MongoDB、Redis、Kafka

**四、大语言模型训练与推理平台 LLMTrain**

**项目背景：**调研并搭建适用于业务场景的 LLM 的训练及推理平台，为 AGI 产品的孵化解决工程问题。

**项目内容：**平台分为训练与推理部分，主要解决大参数量模型在训练和推理中的资源消耗与性能瓶颈。

- 训练平台：基于FastChat框架进行开发，引入Huggingface TRL库扩展PPO、DPO等微调技术，目前训练平台支持SFT（全参数微调）、LoRA、RLHF、DPO等训练模式；
- 推理平台：基于Triton Inference Server框架进行服务管理编排，后端使用vLLM框架进行推理加速。

**个人职责：**负责前期方案调研及整体平台的开发构建

**技术要点：**

- FastChat集成了Transformers和DeepSpeed，利用ZeRO分片机制来降低训练时的显存占用；FastChat支持常见预训练模型的全参数微调和(Q)LoRA微调，通过引入Huggingface TRL库进行二次开发，平台现已能够支持大多数常见模型的RLHF、DPO等复杂微调策略的快速训练；
- Triton Inference Server使用Dynamic batching机制，大幅提升了推理服务的吞吐量和响应速度；后端使用vLLM框架，通过PagedAttention的动态分配机制优化KVCache的内存管理，提升服务稳定性。

**产品落地：**

- AI小右：基于Baichuan-13B，最右应用内智能聊天机器人，主要提供倾听烦恼、陪伴聊天的1v1私信功能，主打幽默风趣的风格，基于最右应用数据进行微调；
- 脱单模拟器：基于Qwen1.5-14B-chat，AI对话挑战产品，通过对话实现男/女神好感度获取以获得青睐；
- 心动Lover：基于Qwen-32b-chat，基于霸道总裁人设的对话闯关产品，创造沉浸式的聊天场景。

**技术组件：**Transformer、FastChat、DeepSpeed、TRL、Triton、vLLM

**五、大语言模型应用开发平台 Argo**

**项目背景：**面向公司内部产品/运营团队及外部客户，提供低代码、可视化的本地化LLM应用开发方案。

**项目内容：**以本地开源模型为主，闭源模型API为辅，结合RAG技术，提供一站式Agent开发工具。

- 模型部分：支持一键从HF、Ollama或ModelScope下载开源模型或导入自行微调过的模型，后端通过数据库与Ollama结合，进行模型的管理与快速部署；
- Agent和RAG部分：基于LangChain工具管理Prompt模板与数据工具的调用；允许用户自行上传文档/表格等内容作为Agent知识库，定制特定领域的大模型应用。

**个人职责：**负责整体后端框架的搭建与Agent部分的开发

**技术要点：**

- 模型联网功能通过Prompt模板与无头浏览器共同实现；用户输入问题后，通过jieba库提取搜索关键词，从搜索内容中爬取有效信息并添加到上下文中，使得模型能根据联网内容进行有效输出；
- 支持用户自行开发内部数据工具，通过模块化的插件形式进行添加；或调用外部服务API来获取外部数据；支持用户输入/模型输出内容审查，以确保Agent工具的合规性；
- 支持代码Sandbox（仅支持Python），使用libseccomp库提供安全的代码运行环境供用户/模型使用；
- RAG知识库使用Milvus作为向量数据库；在按句或按行对用户上传内容进行分割后，使用Bert模型将内容进行向量化；当用户输入查询问题时，使用相同的模型将问题内容向量化并进行相似度查询，将TopN的检索内容与原始问题合并后，形成更丰富的上下文信息，使得大模型的回答内容更加具体且准确。

**技术组件：**Python、Ollama、LangChain、Milvus